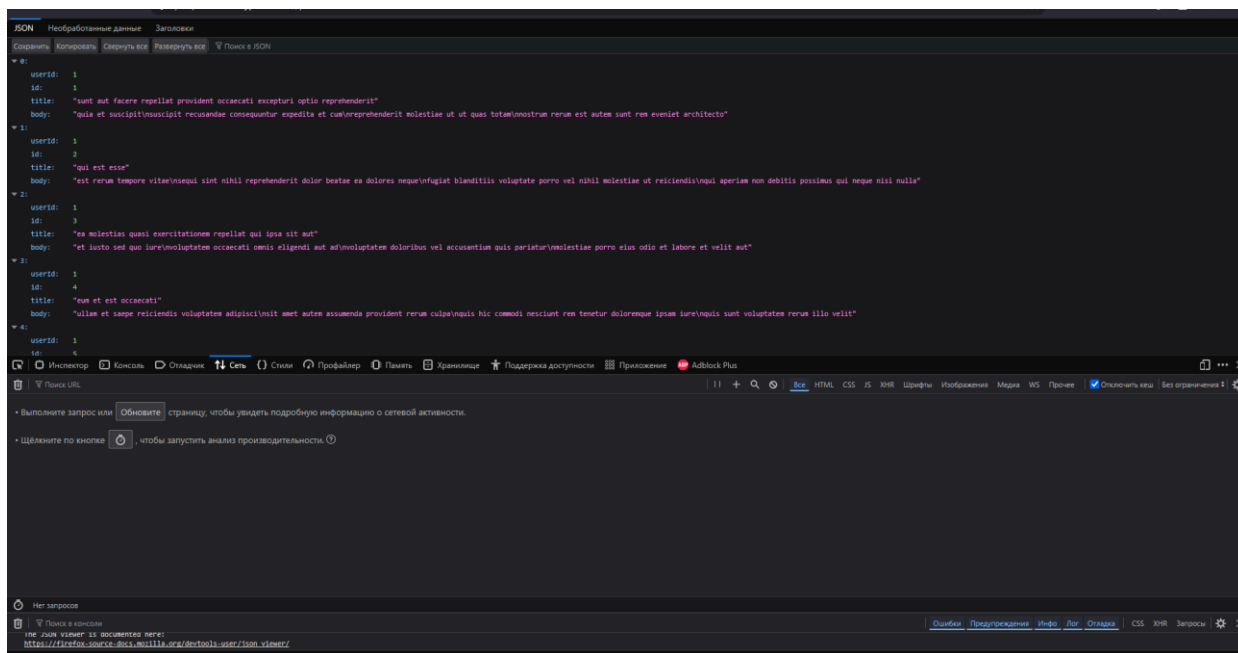


Практика №1

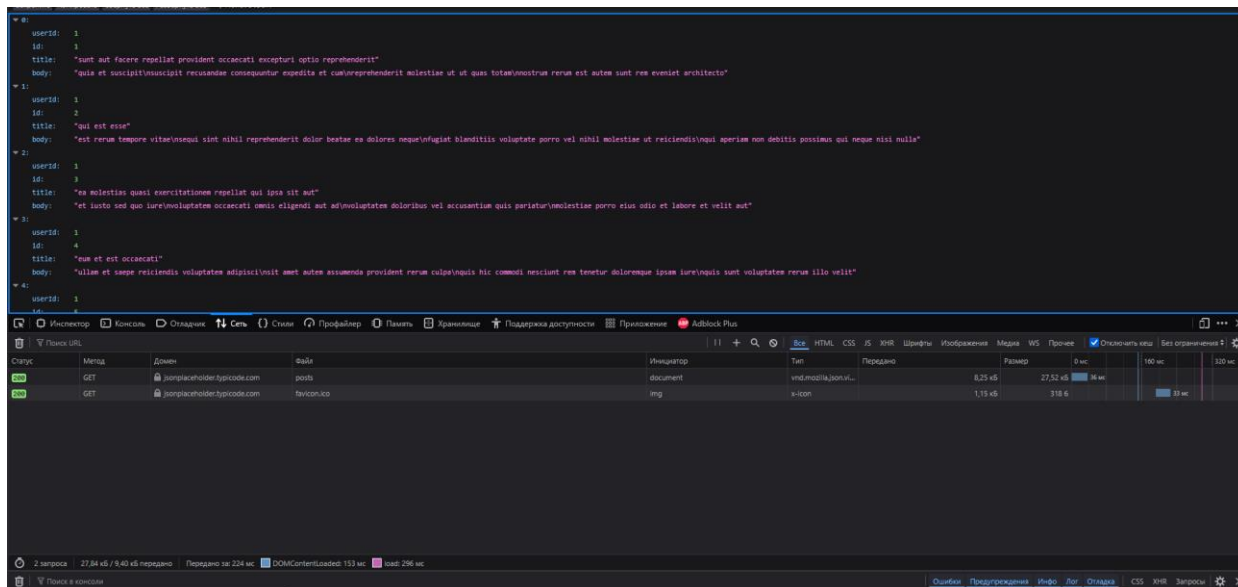
Пользовательские запросы в веб-приложении (DevTools, HTTP, REST, JSON, Postman)

2.1



Posts:

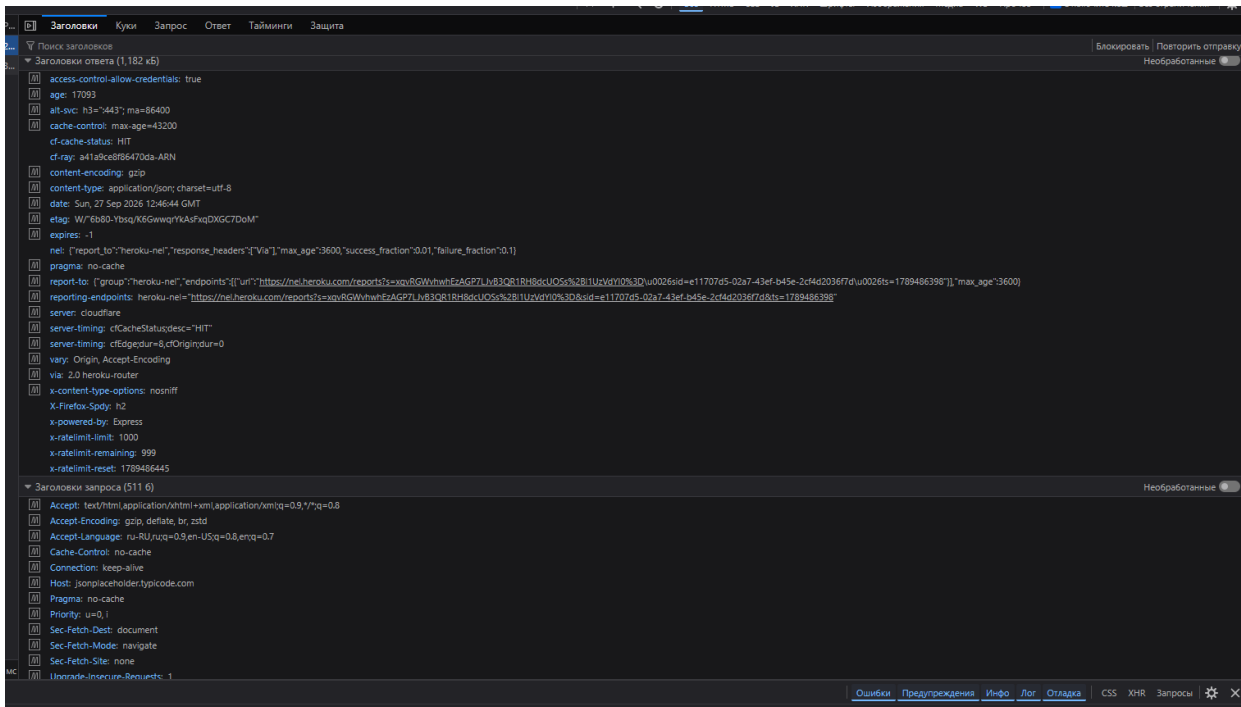
Зашёл в браузер, загрузил страницу — <https://jsonplaceholder.typicode.com/posts>



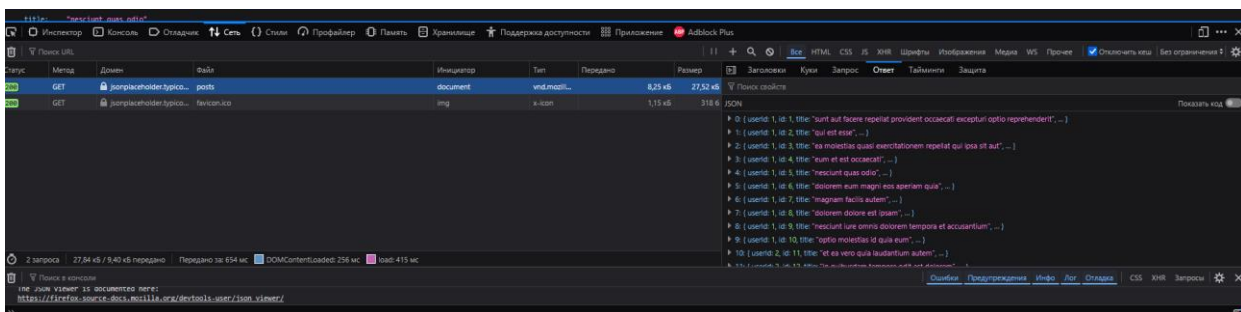
Метод: GET.

url и параметры: <https://jsonplaceholder.typicode.com/posts>, параметров нет.

Заголовки:



Тело ответа:



Код ответа: 200.

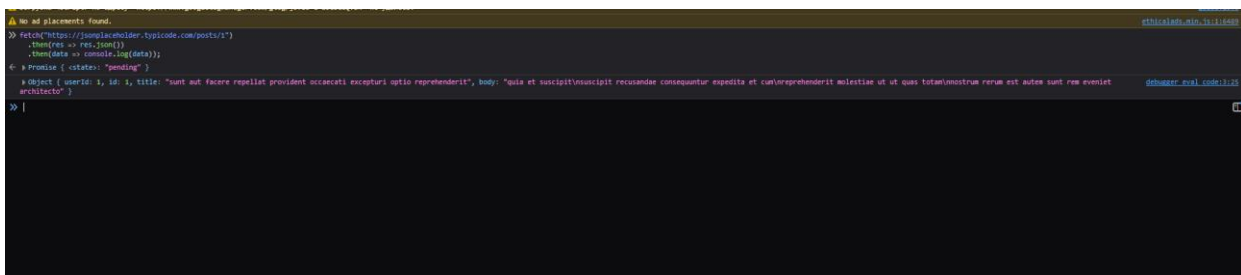
Время выполнения: 36 мс.

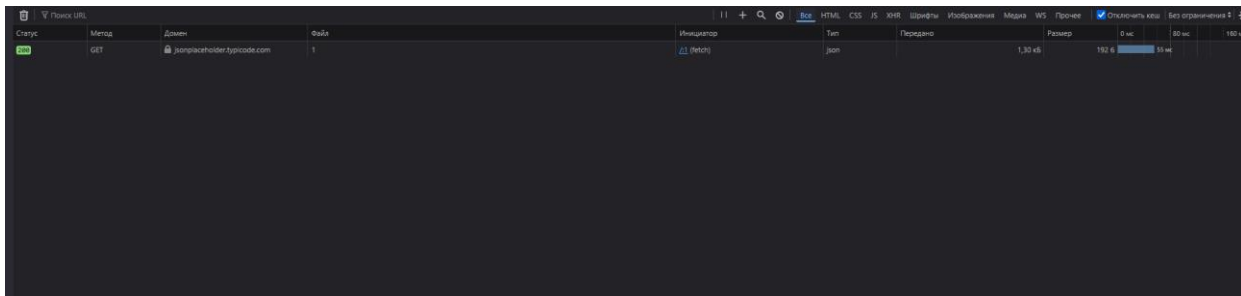
Второй запрос сделал через fetch в консоли (fetch("https://jsonplaceholder.typicode.com/posts/1")):

Метод: GET.

url и параметры: https://jsonplaceholder.typicode.com/posts/1, id передан как часть пути, query-параметров нет.

Инициатор: fetch — не document, как у обычной загрузки страницы.





Тело ответа: объект с `userId`, `id`, `title`, `body`.

Код ответа: 200.

Время выполнения: 55 мс.

Posts:

Размер ответа: 27,52 КБ (передано 8,25 КБ).

Время загрузки: 36 мс.

Количество записей: 100.

Posts/1 (через fetch):

Размер ответа: 192 Б.

Время загрузки: 55 мс.

Количество записей: 1.

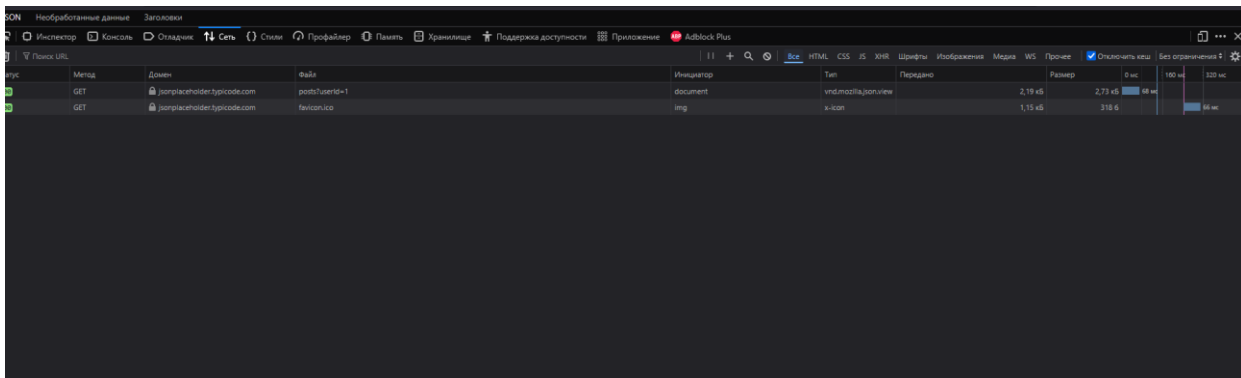
Время загрузки почти одинаковое, а размер отличается в сотни раз — сервер и CDN Cloudflare уже кэшируют `/posts` (в заголовках видно `cf-cache-status: HIT`), поэтому время не всегда прямо пропорционально размеру ответа.

Userid1:

1) 10 объектов вернулось

2) Размер: 2,73 КБ (передано 2,19 КБ)

3) Время: 68 мс



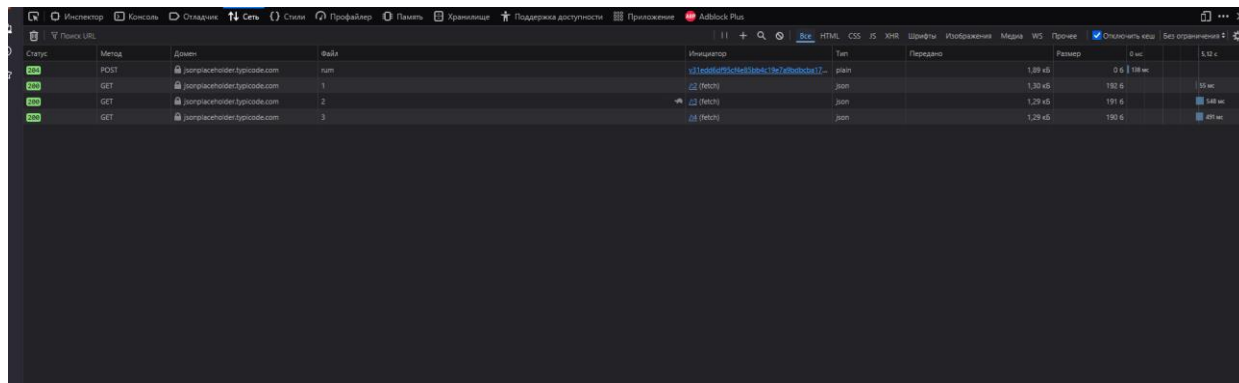
Масс. запросы:

Три запроса /posts/1, /posts/2, /posts/3 запущены одновременно через Promise.all в консоли.

Время отклика: 55 / 548 / 491 мс.

Запросы сработали параллельно — все три стартуют почти в одно и то же время (видно по диаграмме справа в Сети), а не один за другим.

Избыточные запросы: нет, все три URL разные.



Ключевые заголовки в Headers:

content-type - application/json; charset=utf-8

cache-control - max-age=43200

etag - W/"6b80-Ybsq/K6GwwqrYkAsFxqDXGC7DoM"

date - Sun, 27 Sep 2026 12:46:44 GMT

server - cloudflare

cf-cache-status - HIT

Preview и Response: Preview показывает уже распарсенный JSON деревом (удобно для беглого просмотра структуры), Response — сырой текст ответа ровно в том виде, в каком он пришёл по сети (удобно для точности и копирования).

```
JSON Показать код
▶ 0: { userid: 1, id: 1, title: "sunt aut facere repellat provident occaecati excepturi optio reprehenderit", ... }
▶ 1: { userid: 1, id: 2, title: "qui est esse", ... }
▶ 2: { userid: 1, id: 3, title: "ea molestias quasi exercitationem repellat qui ipsa sit aut", ... }
▶ 3: { userid: 1, id: 4, title: "eum et est occaecati", ... }
▶ 4: { userid: 1, id: 5, title: "nesciunt quas odio", ... }
▶ 5: { userid: 1, id: 6, title: "dolorem eum magni eos aperiam quia", ... }
▶ 6: { userid: 1, id: 7, title: "magnam facilis autem", ... }
▶ 7: { userid: 1, id: 8, title: "dolorem dolore est ipsam", ... }
▶ 8: { userid: 1, id: 9, title: "nesciunt iure omnis dolorem tempora et accusantium", ... }
▶ 9: { userid: 1, id: 10, title: "optio molestias id quia eum", ... }
▶ 10: { userid: 2, id: 11, title: "et ea vero quia laudantium autem", ... }
▶ 11: { userid: 2, id: 12, title: "in quibusdam tempore odit est dolorem", ... }
▶ 12: { userid: 2, id: 13, title: "dolorum ut in voluptas mollitia et saepe quo animi", ... }
▶ 13: { userid: 2, id: 14, title: "voluptatem eligendi optio", ... }
▶ 14: { userid: 2, id: 15, title: "eveniet quod temporibus", ... }
▶ 15: { userid: 2, id: 16, title: "sint suscipit perspiciatis velit dolorum rerum ipsa laboriosam odio", ... }
▶ 16: { userid: 2, id: 17, title: "fugit voluptas sed molestias voluptatem provident", ... }
▶ 17: { userid: 2, id: 18, title: "voluptate et itaque vero tempora molestiae", ... }
▶ 18: { userid: 2, id: 19, title: "adipisci placeat illum aut reiciendis qui", ... }
▶ 19: { userid: 2, id: 20, title: "doloribus ad provident suscipit at", ... }
▶ 20: { userid: 3, id: 21, title: "asperiores ea ipsam voluptatibus modi minima quia sint", ... }
▶ 21: { userid: 3, id: 22, title: "dolor sint quo a velit explicabo quia nam", ... }
▶ 22: { userid: 3, id: 23, title: "maxime id vitae nihil numquam", ... }
▶ 23: { userid: 3, id: 24, title: "autem hic labore sunt dolores incidunt", ... }
▶ 24: { userid: 3, id: 25, title: "rem alias distinctio quo quis", ... }
▶ 25: { userid: 3, id: 26, title: "est et quae odit qui non", ... }
▶ 26: { userid: 3, id: 27, title: "quasi id et eos tenetur aut quo autem", ... }
▶ 27: { userid: 3, id: 28, title: "delectus ullam et corporis nulla voluptas sequi", ... }
▶ 28: { userid: 3, id: 29, title: "iusto eius quod necessitatibus culpa ea", ... }
▶ 29: { userid: 3, id: 30, title: "a quo magni similique perferendis", ... }
▶ 30: { userid: 4, id: 31, title: "ullam ut quidem id aut vel consequuntur", ... }
▶ 31: { userid: 4, id: 32, title: "doloremque illum aliquid sunt", ... }
▶ 32: { userid: 4, id: 33, title: "qui explicabo molestiae dolorem", ... }
▶ 33: { userid: 4, id: 34, title: "magnam ut rerum iure", ... }
▶ 34: { userid: 4, id: 35, title: "id nihil consequatur molestias animi provident", ... }
▶ 35: { userid: 4, id: 36, title: "fuga nam accusamus voluptas reiciendis itaque", ... }
▶ 36: { userid: 4, id: 37, title: "provident vel ut sit ratione est", ... }
```

```
JSON Показать код
1 {
2   {
3     "userid": 1,
4     "id": 1,
5     "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
6     "body": "quis et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas tota\nnostrum rerum est autem sunt eveniet architecto"
7   },
8   {
9     "userid": 1,
10    "id": 2,
11    "title": "qui est esse",
12    "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis possimus qui neque nisi nulla"
13  },
14  {
15    "userid": 1,
16    "id": 3,
17    "title": "ea molestias quasi exercitationem repellat qui ipsa sit aut",
18    "body": "et iusto sed quo iure\nvoluptatem occaecati omnis eligendi ad ad\nvoluptatem doloribus vel accusantium quis pariatur\nmolestiae porro eius odio et labore et velit aut"
19  },
20  {
21    "userid": 1,
22    "id": 4,
23    "title": "eum et est occaecati",
24    "body": "ullam et saepe reiciendis voluptatem adipisci\nsit amet autem assumenda provident rerum culpa\nquis hic commodi nesciunt rem tenetur doloremque ipsam iure\nquis sunt voluptatem rerum illo velit"
25  },
26  {
27    "userid": 1,
28    "id": 5,
29    "title": "nesciunt quas odio",
30    "body": "repudiandae veniam quaerat sunt sed\nnihil aut fugiat sit autem sed est\nvoluptatem omnis possimus esse voluptatibus quis\nnest aut tenetur dolor neque"
31  },
32  {
33    "userid": 1,
34    "id": 6,
35    "title": "dolorem eum magni eos aperiam quia",
36    "body": "ut aspernatur corporis harum nihil quis provident sequi\nmollitia nobis aliquid molestiae\nperspiciatis et ea nemo ab reprehenderit accusantium quas\nvoluptate dolores velit et doloremque molestiae"
37  },
38  {
39    "userid": 1,
40    "id": 7,
41    "title": "magnam facilis autem",
42    "body": "dolore placeat quibusdam ea quo vitae\nmagni quis enim qui quis quo nemo aut saepe\nquidem repellat excepturi ut quia\nsunt ut sequi eos ea sed quas"
43  },
44  {
45    "userid": 1,
46    "id": 8,
47    "title": "dolorem dolore est ipsam",
```

2.8. Оптимизация запросов

Сравнил три запроса: `/posts`, `/posts?userId=1`, `/posts/1`.

Запрос `/posts` — размер ответа 27,52 КБ, время загрузки 36 мс, количество записей 100.

Запрос `/posts?userId=1` — размер ответа 2,73 КБ, время загрузки 68 мс, количество записей 10.

Запрос `/posts/1` — размер ответа 192 Б (fetch, 55 мс) / 1,31 КБ (Postman, 22 мс), количество записей 1.

Вывод: параметры и точечные запросы уменьшают объём данных в разы. Время загрузки не всегда прямо пропорционально размеру ответа — сказываются кэш CDN и случайные сетевые задержки.

Меры оптимизации:

Пагинация — грузить по 10–20 записей вместо 100.

Фильтрация — по `userId`, `id`, статусу.

Убрать лишние поля из ответа.

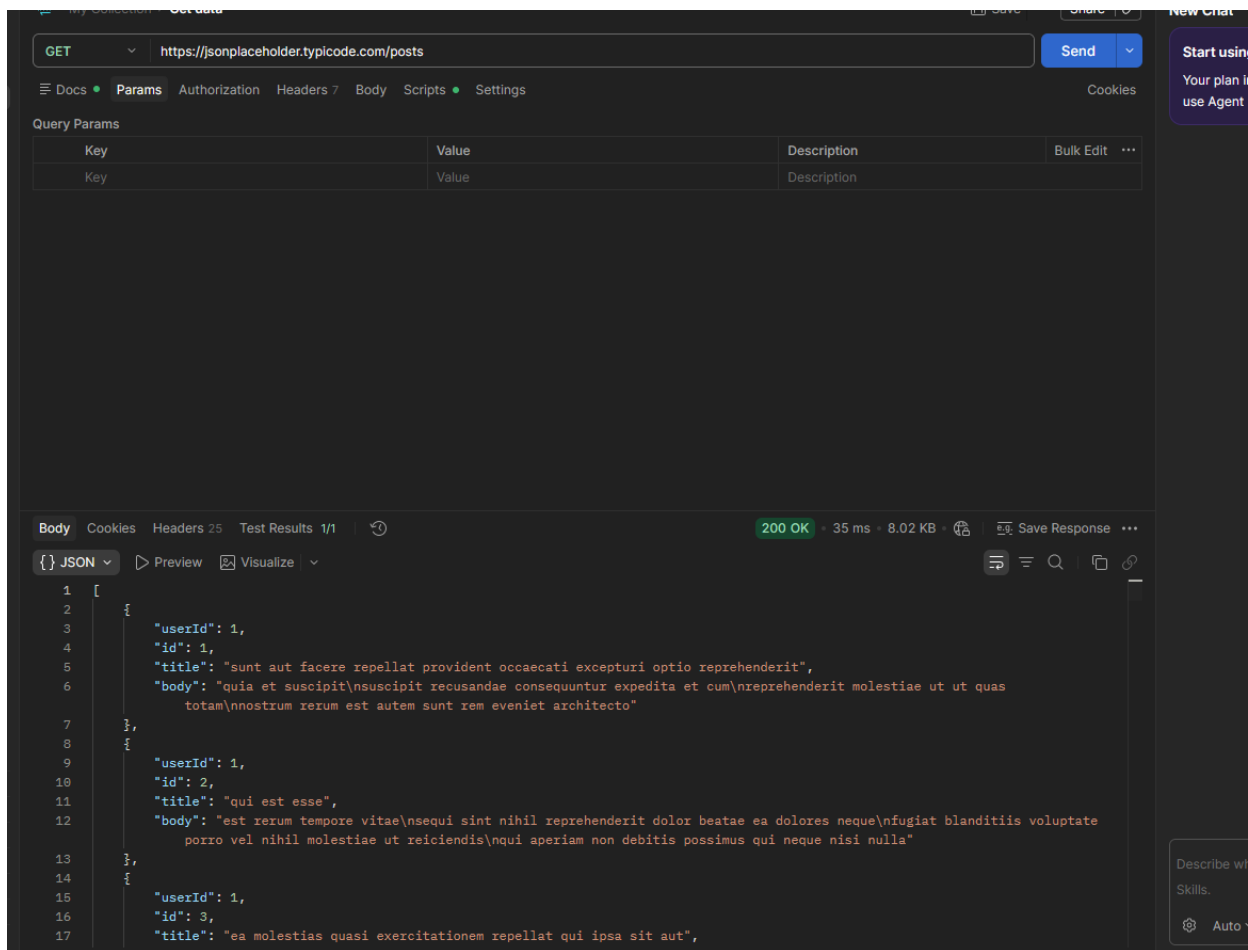
Кэширование на клиенте и сервере (Cache-Control, ETag).

Сжатие ответа (gzip) — уже применяется сервером.

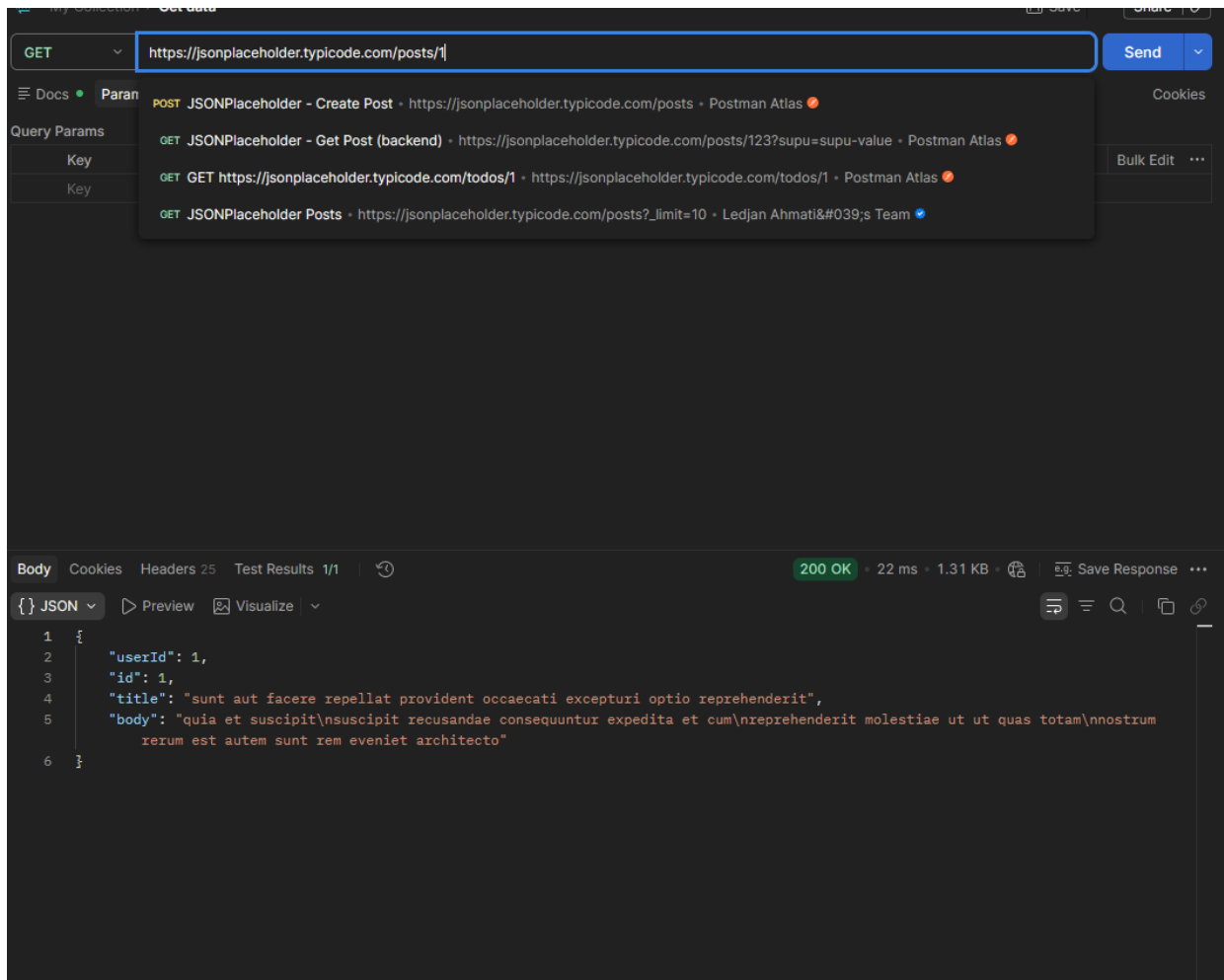
2.9. Работа с API через Postman

Зашёл на postman.com, создал новый workspace.

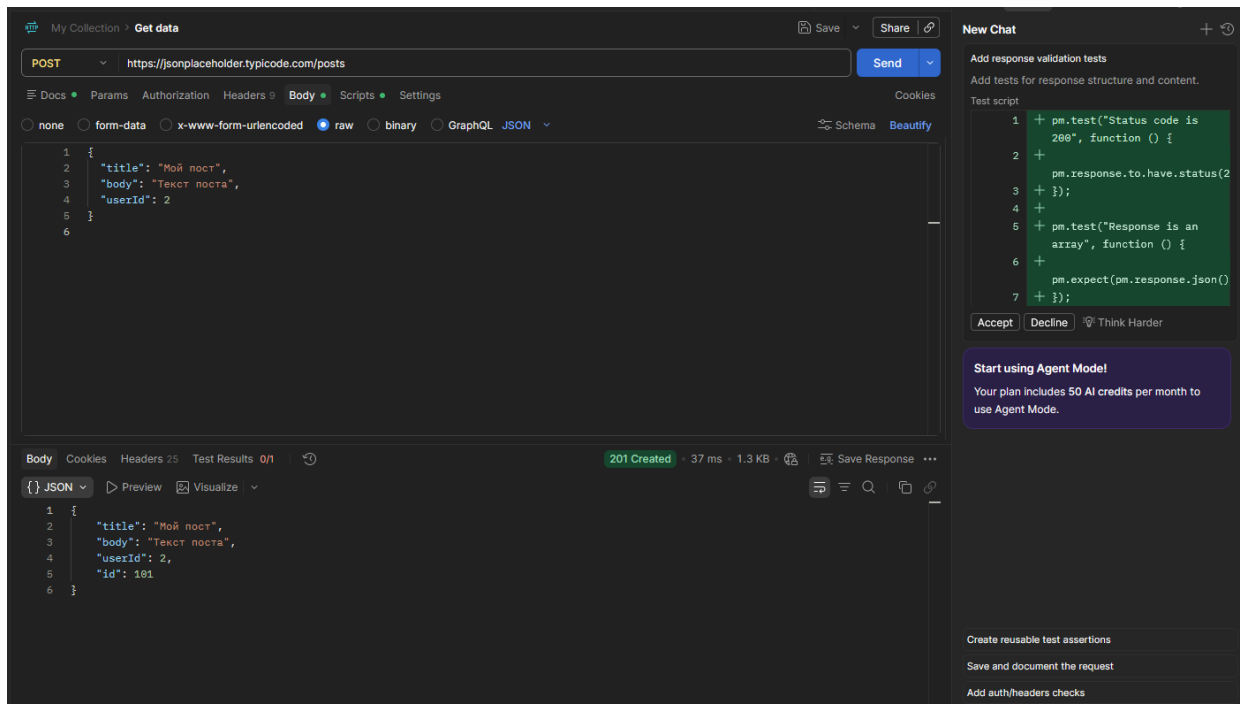
GET `/posts` — вернул 100 записей, размер 8,02 КБ, код 200, 35 мс.



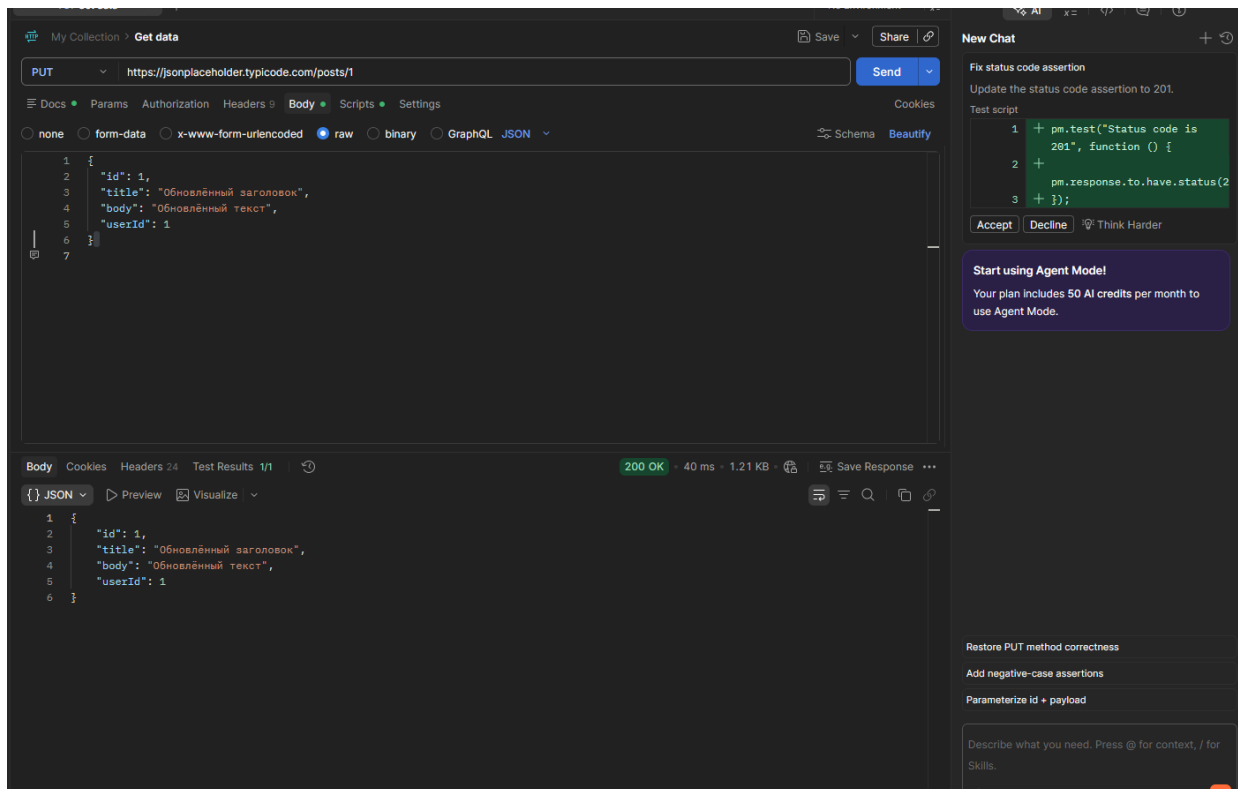
GET /posts/1 — вернул 1 запись, размер 1,31 КБ, код 200, 22 мс.



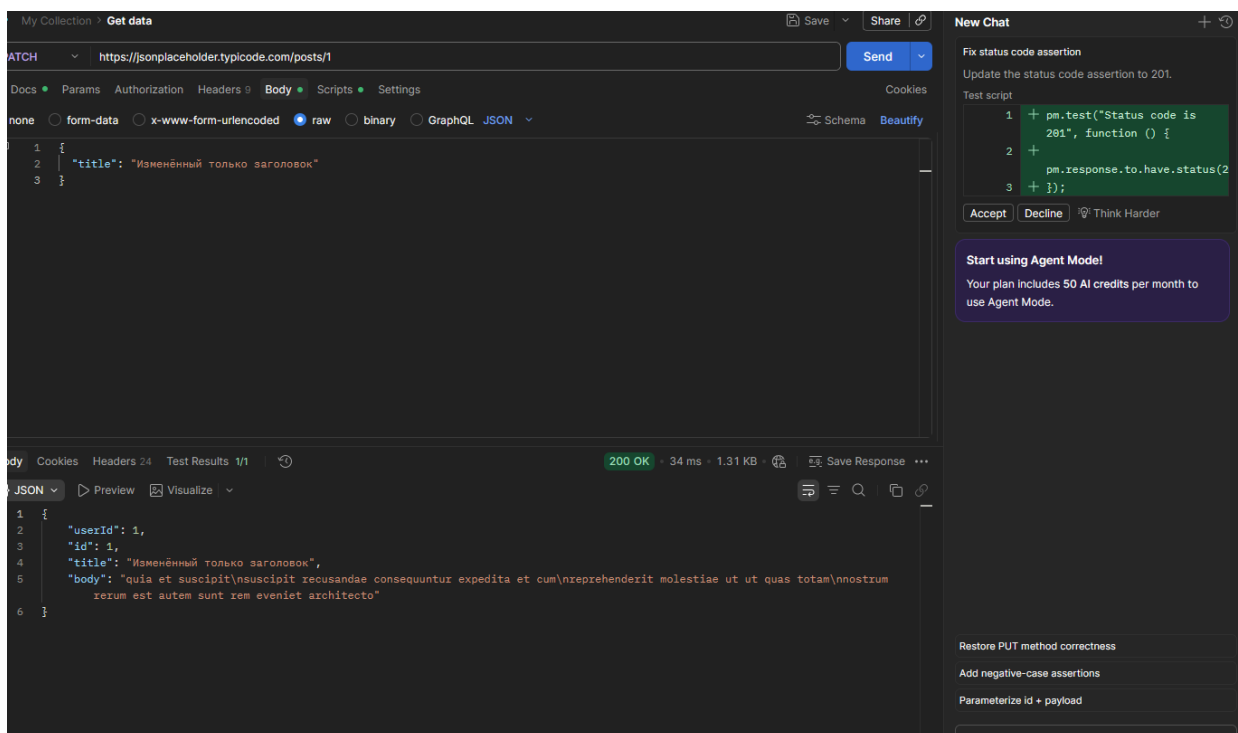
POST /posts — отправил JSON с title, body и userId. Сервер вернул код 201 Created, размер 1,3 КБ, время 37 мс, и созданный объект с id 101.



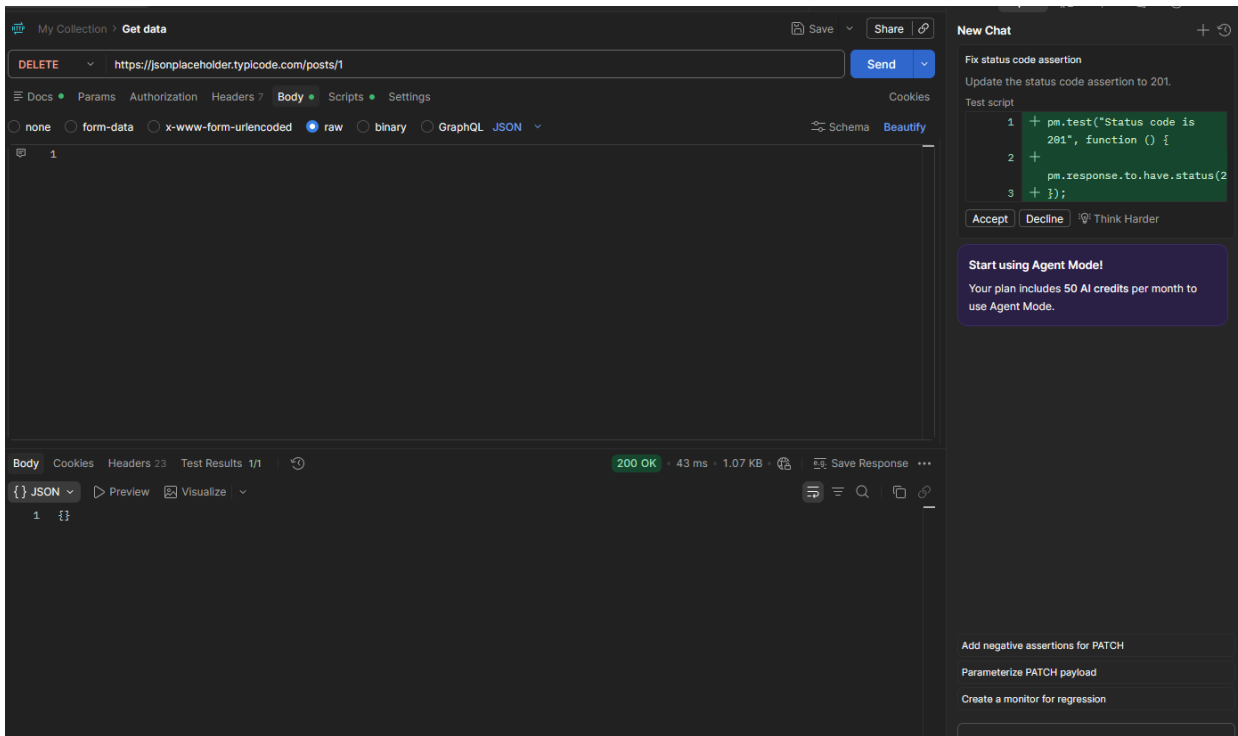
PUT /posts/1 — полное обновление. Отправил id, title, body, userId. Вернулся код 200, размер 1,21 КБ, время 40 мс, обновлённый объект.



PATCH /posts/1 — частичное обновление. Изменил только title. Вернулся код 200, размер 1,31 КБ, время 34 мс, объект с изменённым title и сохранёнными старыми body и userId.



DELETE /posts/1 — удаление. Код ответа 200, размер 1,07 КБ, время 43 мс. Тело ответа пустое {}.



Разница между PUT и PATCH: PUT заменяет объект целиком (в ответе только отправленные поля), PATCH меняет только указанные поля, остальные сохраняются из исходного объекта.

2.10. Сравнение DevTools и Postman

DevTools — анализ реальных запросов уже открытой страницы. Только просмотр. Коллекций нет, тестов нет. Авторизация — просмотр уже применённых токенов. Есть замеры производительности, кэш, тайминги.

Postman — ручное конструирование запросов независимо от сайта. Создание, отправка, сохранение. Коллекции есть, тесты есть. Авторизация настраивается полностью. Производительность страницы не показывает.

Вывод: DevTools удобен для отладки сайта и анализа того, что уже происходит. Postman нужен для тестирования API, работы со всеми методами и сохранения коллекций запросов.

3. Контрольные вопросы

1. Что такое DevTools и какие задачи решают вкладки Elements, Console и Network?

DevTools — инструменты разработчика в браузере. Elements показывает DOM и структуру страницы. Console выводит ошибки и логи, позволяет выполнять код. Network анализирует запросы, коды ответов, заголовки и время.

2. Как открыть DevTools и какие настройки вкладки Network важны для замеров производительности?

Открыть по F12 или правой кнопкой — «Исследовать». Важные настройки: режим Online (без ограничений скорости), отключение кэша, фильтр по типу запросов, сортировка по времени.

3. Перечислите основные части HTTP-запроса.

Метод, URL и параметры, заголовки, тело запроса, тело ответа, код ответа, время выполнения.

4. Чем отличаются методы GET, POST, PUT, PATCH и DELETE?

GET — получить данные. POST — создать. PUT — заменить целиком. PATCH — изменить частично. DELETE — удалить. Пример: GET /posts/1, POST /posts, PUT /posts/1, PATCH /posts/1, DELETE /posts/1.

5. Для чего используются методы HEAD и OPTIONS?

HEAD — получить только заголовки без тела. OPTIONS — узнать, какие методы поддерживает сервер. Preflight-запрос встречается при CORS, когда браузер сначала сам отправляет OPTIONS перед основным запросом.

6. Расскажите о группах кодов ответов.

1xx — информационные. 2xx — успешные (200, 201, 204). 3xx — перенаправления (301, 302, 304). 4xx — ошибки клиента (400, 401, 403, 404). 5xx — ошибки сервера (500, 502, 503).

7. Что такое REST и как ресурс в REST представлен на уровне URL?

REST — архитектурный стиль взаимодействия с ресурсами по HTTP. Ресурс представлен уникальным URL, например /posts/1, а действие над ним задаёт HTTP-метод.

8. Какие типы данных поддерживает JSON?

Объекты, массивы, строки, числа, boolean, null. Пример: {"userId": 1, "id": 1} и [1, 2, 3].

9. В чём отличие вкладок Preview и Response?

Preview показывает данные в удобном виде: JSON раскрывается деревом. Response показывает сырой текст. Preview удобен для чтения, Response — для проверки исходного ответа.

10. Какие заголовки чаще всего анализируются при оптимизации?

Content-Type, Cache-Control, ETag, Date, Server, Authorization.

11. Какие приёмы снижения нагрузки на сеть можно использовать для API?

Пагинация, фильтрация, кэширование, сжатие, удаление лишних полей, правильные заголовки.

12. Чем работа с API в Postman отличается от анализа запросов в DevTools?

Postman позволяет создавать запросы вручную, сохранять коллекции, писать тесты. DevTools показывает реальные запросы уже открытой страницы и помогает с отладкой.